WikiPulse: Real-Time Analysis of Wikipedia Edit Streams

ID2221 final project · Muyang Huang, Yanjun Wang, Hao Tang · code, run instructions and raw results in this repository

1. What we built

WikiPulse is an end-to-end streaming platform for the Wikimedia `recentchange` feed. It runs as one Docker Compose stack on a laptop: Kafka 3.9, HDFS 3.2, Spark 4.2 with Delta 4.4, Cassandra 4.1 and Grafana 12.2.

- **Ingest.** A Python producer reads the SSE feed and writes every raw event to Kafka. The Kafka key is the Wikimedia `meta.id`. The producer saves the SSE position only after Kafka acknowledges the write, so delivery is at least once.
- **Archive.** A Structured Streaming job archives the unfiltered stream to an HDFS Delta table, keeping payload, headers and offsets.
- **Speed layer.** B's Structured Streaming job applies the shared rules in `events.py`: Wikipedia articles only, `type` ∈ {edit, new}, a 2-minute event-time watermark, and de-duplication by source ID. It runs three append-mode queries:
  - trending pages in 10-minute windows sliding by 1 minute, scored `edit_count·ln(1+editors)`;
  - edit-war candidates: at least 3 non-bot reverts by at least 2 users with an A→other→A alternation in 30 minutes;
  - 5-minute activity and bot/human counts for every edition, with a z-score above 3 against the baseline flagged as an anomaly.
- **Batch layer.** A Spark SQL job computes the same 5-minute counts from one pinned Delta version. It then publishes the mean and standard deviation per (wiki, UTC hour of day) as an immutable baseline version. A running stream reads one fixed version and switches only on a controlled restart.
- **Serving.** Results are written to Cassandra tables partitioned by `(wiki, day)` or `(day)`. A Grafana dashboard reads them through the Cassandra plugin: trending pages, edit-war candidates, bot/human ratio and share, language share, baseline and activity heatmaps, and anomalies.

No Spark 4 Cassandra connector exists, so `foreachBatch` collects each completed micro-batch on the driver and writes it with prepared INSERTs. Primary-key overwrites make retried batches idempotent.

2. Data

The live capture had stopped after the first 8-hour handoff, so we rebuilt seven days of history from EventStreams, which keeps about a week. We resumed with a timestamp-positioned `Last-Event-ID` for each data centre and grouped Kafka acknowledgements. This raised backfill throughput from about 30 to about 1,800 events/s.

- **Archive.** The pinned archive (Delta version 348) holds 20.4 M raw events from 2026-10-17 12:00 to 10-24 18:15 UTC, 2.9 M per day. Kafka offsets and Delta rows agree exactly.
- **Analysis subset.** Wikipedia projects are 28.9% of the raw stream, and article edits and new pages are 11.9% (2.43 M rows, 335 editions).
- **Baseline.** Baseline `20261023T1200Z` uses six days with no collection gap. It has 72 samples for every (wiki, hour) cell.
- **Event order.** Backfill order differs from event order: the sparse codfw stream runs days ahead of eqiad. A replay in fetch order would push the watermark ahead and drop every later event as late. We therefore replay backfilled data in event-time order.

3. Evaluation

All experiments used the Compose stack as deployed: a single Spark worker with 1 core and 1 GiB, a 768 MiB driver and 8 shuffle partitions. Replays read the same pinned archive slice starting at 2026-10-23 12:00, after the baseline history. The natural rate of that slice is about 34 raw events/s.

Throughput and latency. A's replay sends the same 50,000 archive rows into a fresh topic at a fixed rate. We define window latency as Cassandra `written_at` minus the Kafka append time of the first event that made the window closable, i.e. the first event whose event time passes `window_end + 2 min`. It therefore includes trigger wait and processing but not the watermark wait, which is part of the window semantics. Each configuration ran twice.

Kafka partitions	Rate (events/s)	Trigger	Hot pages latency p50 / p95	Activity latency p50
3	100 / 200 / 400	5 s	17 / 26 · 18 / 31 · 17 / 22 s	28 · 37 · 27 s
3	100 / 200 / 400	30 s	48 / 60 · 51 / 64 · 54 / 64 s	47 · 48 · 45 s
3 vs 1	200	10 s	19 / 25 vs 19 / 24 s	25 vs 15 s

Every paced configuration kept up with its input: throughput equalled the offered rate, and the last batch ended within 34 s of the replay. Draining 50,000 preloaded rows in 10,000-row batches reached 634–737 events/s, about 20× the natural rate.

- **Trigger interval, not input rate, sets latency.** The window is emitted only by the batch after the one that advances the watermark. A 30 s trigger therefore costs about 50 s.

- **Short triggers run back to back.** At 5–10 s, a batch of three stateful queries takes 7–12 s on the single core, so latency is about two batch durations (17–19 s for hot pages).
- **Kafka partitions had no measurable effect** with one executor core. The spread between repetitions was as large as any partition effect.
- **The live bottleneck was the producer.** Waiting for every Kafka acknowledgement caps the live producer at about 32 events/s, below the evening feed. Its lag grew to 15 minutes in an hour. Grouped acknowledgements (`--ack-every`) remove that limit while staying at least once.

Correctness.

- A batch recount of a replay topic with the same rules matched every closed `activity_5m` window: 198 of 198 windows and 8,821 edits.
- In the live run, we killed the streaming driver and restarted it from its checkpoint. It resumed with contiguous 5-minute windows and no duplicates.
- Live detection flagged an enwiki revert burst: 151 reverts in 5 minutes against a baseline of about 17, z = 16.7.

Edit-war precision. The heuristic flagged 1,052 windows on 209 pages in seven days. We labelled a seeded random sample of 50 pages from edit summaries, using B's rubric: a positive is a substantive dispute over content or sourcing.

- Results: 8 positive, 33 negative and 9 uncertain. Precision is **19.5%** (8/41, Wilson 95% CI 10–34%), or 16% if uncertain pages count as negative.
- The false positives are anti-vandalism work that satisfies the same alternation pattern: patrol-tool reverts (14), vandalism rollbacks (14) and sockpuppet cleanup (5).
- The dashboard therefore labels these pages as *candidates*. Filtering on revert summaries that name tools, vandalism or blocks, or on temporary accounts, is the obvious next refinement.

Delta partition pruning. The archive is partitioned by ingest date, so the whole backfill sits in one partition and pruning cannot help it. We copied version 348 into a table partitioned by event date and ran the same one-day 5-minute count job three times each way, with Delta data skipping disabled. Outputs were identical.

	Files read	File bytes	Median time
Partition filter on	292	9.4 GB	51.4 s
Event-time range only	4,580	71.3 GB	60.9 s

Pruning skipped 94% of the files but saved only 16% of the time. The unpruned scan already avoids most of the payload through Parquet row-group min/max filtering, because the files are close to event-time order. The rest of the job's cost does not depend on how much is scanned. Partitioning by event date matters most for scans that cannot be pushed down to row groups, and for keeping the planning and listing cost proportional to the requested range.

4. Limitations and lessons

- **Compute.** One 1-core executor sustains about 20× the natural rate, but leaves no headroom for larger state or for running batch jobs on the cluster. We ran the batch jobs in local mode on the client container.
- **Latency.** On this hardware a 5–10 s trigger gives about 18 s for trending pages; longer triggers only add delay, since even 5 s keeps up at 12× the natural rate.
- **Labels.** The precision estimate comes from one AI-assisted rater using edit summaries, without a second review, and the sample has few non-English pages.
- **Partitioning.** Partitioning the raw archive by ingest date is right for append and retention but wrong for historical batch jobs. A compacted event-date copy is the better input for baselines.